
NetSentry : Développement d'un outil de découverte et de cartographie des équipements connectés au réseau informatique

MARWANE AMAL

2^{ÈME} ANNÉE CYCLE D'INGÉNIEUR
(4IIR)
INGÉNIERIE INFORMATIQUE ET RÉSEAUX

Septembre 2026

Encadrant :
BAZZA HOUSSAM

Membres du jury :
BAZZA HOUSSAM
ABROUN SOUNDOUSS

Dédicace

À mes chers parents, pour leur soutien indéfectible, leurs sacrifices et leurs encouragements tout au long de mon parcours.

À mes professeurs et formateurs, pour la qualité de l'enseignement transmis et pour m'avoir accompagné dans la construction de mes compétences.

À mes proches et amis, pour leur présence et leurs encouragements constants.

Je dédie ce modeste travail.

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes ayant contribué, de près ou de loin, à la réussite de ce stage.

Je remercie tout particulièrement mon encadrant pédagogique au sein de l'École Marocaine des Sciences de l'Ingénieur (EMSI) de Tanger, pour son suivi régulier, ses conseils avisés et sa disponibilité tout au long de la réalisation de ce projet.

Mes remerciements s'adressent également à l'ensemble du département Informatique (IT) du Mövenpick Hôtel de Tanger, et en particulier à mon encadrant professionnel Anas, pour m'avoir accueilli au sein de son équipe et avoir confié un projet concret et formateur.

Je souhaite aussi remercier l'ensemble du corps professoral et administratif de l'EMSI Tanger, pour la qualité de la formation dispensée durant ces années, qui m'a fourni les bases nécessaires à la conception et à la réalisation de ce projet.

Enfin, je remercie ma famille et mes proches pour leur soutien constant durant cette période de stage.

Résumé

Contexte général. Ce rapport s’inscrit dans le cadre d’un stage de perfectionnement de deux mois effectué au sein du département Informatique (IT) du Mövenpick Hôtel de Tanger. Le réseau local de l’hôtel voit se connecter, au fil du temps, un grand nombre d’équipements (postes fixes, imprimantes, terminaux mobiles, objets connectés), sans que le département IT dispose d’un moyen simple d’en garder un inventaire à jour.

Objectif du projet. L’objectif du projet est de développer un outil de découverte et de cartographie des équipements connectés au réseau local de l’hôtel, baptisé **NetSentry**, capable de scanner le réseau, d’identifier les équipements actifs (adresse IP, adresse MAC, fabricant, ports ouverts), de conserver un historique des scans, et de signaler automatiquement tout équipement inconnu apparaissant sur le réseau.

Méthode. La réalisation du projet s’appuie sur une mise à niveau sur les fondamentaux réseau (adressage IP/MAC, protocole ARP, notion de ports), puis sur la conception et le développement d’une architecture en trois couches : un moteur de scan reposant sur Nmap, une API backend (Node.js/Express) assurant l’orchestration, la persistance (PostgreSQL) et la détection d’anomalies, ainsi qu’un tableau de bord web (React) permettant à un administrateur IT de visualiser les équipements, l’historique des scans et les alertes. La solution est conteneurisée avec Docker afin de faciliter son déploiement.

Résultats. L’outil développé permet de scanner le réseau local, de recenser les équipements connectés avec leur fabricant et leurs ports ouverts, de conserver un historique complet des scans, et de générer une alerte dès qu’un équipement non reconnu se connecte au réseau. Le fonctionnement de l’ensemble de la chaîne – scan, backend, tableau de bord et déploiement conteneurisé – a été vérifié de bout en bout durant le stage.

Mots-clés : Découverte réseau, Cartographie d’équipements, Nmap, Détection d’anomalies, Tableau de bord, Docker, NetSentry.

Abstract

Context. This report is part of a two-month internship carried out within the IT department of the Mövenpick Hôtel Tanger. Over time, a large number of devices connect to the hotel's local network (workstations, printers, mobile terminals, connected devices), without the IT department having a simple way to keep an up-to-date inventory of them.

Objective. The objective of the project is to develop a network device discovery and mapping tool, named **NetSentry**, able to scan the hotel's local network, identify active devices (IP address, MAC address, vendor, open ports), keep a history of scans, and automatically flag any unknown device appearing on the network.

Method. The project relies on an initial upskilling phase covering network fundamentals (IP/MAC addressing, the ARP protocol, ports), followed by the design and development of a three-layer architecture : an Nmap-based scanning engine, a backend API (Node.js/Express) handling orchestration, persistence (PostgreSQL) and anomaly detection, and a web dashboard (React) allowing an IT administrator to view devices, scan history and alerts. The solution is containerized with Docker to ease deployment.

Results. The resulting tool scans the local network, lists connected devices along with their vendor and open ports, keeps a full history of scans, and raises an alert whenever an unrecognized device joins the network. The full chain – scanning, backend, dashboard and containerized deployment – was verified end to end during the internship.

Keywords : Network discovery, Device mapping, Nmap, Anomaly detection, Dashboard, Docker, NetSentry.

Table des matières

Dédicace	1
Remerciements	2
Résumé	3
Abstract	4
Table des figures	7
Liste des abréviations	8
Introduction	9
1 Contexte général	11
1 Introduction	11
2 Organisme d'accueil	11
2.1 Présentation de l'entreprise	11
2.2 Environnement de l'entreprise	12
2.3 Département d'accueil et mission	12
3 Contexte	12
4 Problématique	13
5 Objectifs du projet	13
6 Solutions existantes	13
6.1 Outils de scan en ligne de commande	14
6.2 Applications grand public	14
6.3 Solutions de supervision professionnelles	14
7 Limites observées	14
8 Analyse SWOT	15
9 Méthodologie de travail	15
10 Conclusion	15

2	Analyse des besoins et conception	17
1	Introduction	17
2	Cahier des charges	17
2.1	Livrables attendus	17
2.2	Contraintes	17
3	Exigences fonctionnelles	18
4	Exigences non fonctionnelles	18
5	Diagramme de cas d'utilisation	19
6	Diagramme de séquence	21
7	Modélisation des données (MCD)	22
8	Conclusion	23
3	Architecture technique et technologies	24
1	Introduction	24
2	Nmap : l'outil au cœur du moteur de scan	24
2.1	Présentation	24
2.2	Découverte d'hôtes par ARP	25
2.3	Les deux phases du scan NetSentry	25
2.4	Choix hors périmètre	26
2.5	Identification du fabricant (OUI)	26
3	Autres technologies utilisées	26
3.1	Backend	26
3.2	Base de données	27
3.3	Frontend	27
3.4	Conteneurisation et versionnage	27
4	Architecture du système	28
4.1	Architecture en couches	28
5	Planification du projet	30
5.1	Diagramme de Gantt	30
5.2	Organisation du travail (Kanban)	30
6	Conclusion	31
4	Mise en œuvre et captures d'écran	32
1	Introduction	32
2	Tableau de bord	32
3	Détail d'un appareil	33
4	Alertes	34
5	Historique des scans	34
6	Versionnage	35
7	Conclusion	35

Conclusion Générale	36
1 Bilan des réalisations	36
2 Défis rencontrés et solutions apportées	36
3 Compétences acquises et renforcées	37
4 Perspectives d'évolution	37

Table des figures

2.1 Diagramme de cas d'utilisation de NetSentry : l'authentification est incluse par l'ensemble des actions de l'administrateur IT	20
2.2 Diagramme de séquence – exécution d'un scan et détection d'une anomalie	21
2.3 Modèle conceptuel de données (MCD) de NetSentry	22
3.1 Logo du projet Nmap (nmap.org)	24
3.2 Technologies du backend NetSentry	26
3.3 Logo PostgreSQL	27
3.4 Logo React	27
3.5 Logo Docker	28
3.6 Logo Git	28
3.7 Architecture générale de NetSentry et intégration au réseau local de l'hôtel	29
3.8 Diagramme de Gantt du planning prévisionnel du stage	30
3.9 Organisation du travail selon la méthode Kanban	31
4.1 Tableau de bord : vue d'ensemble des appareils détectés sur le réseau	33
4.2 Détail d'un appareil : historique de détection et mise en liste blanche	33
4.3 Alertes : appareils non reconnus signalés après un scan	34
4.4 Historique des scans exécutés (manuels et planifiés)	34
4.5 Historique des commits du dépôt GitHub NetSentry	35

Liste des abréviations

TABLE 1 – Liste des abréviations utilisées dans le rapport

Abréviation	Signification
API	Application Programming Interface (interface de programmation applicative)
ARP	Address Resolution Protocol (protocole de résolution d'adresse)
CIDR	Classless Inter-Domain Routing (notation de plage d'adresses réseau)
CLI	Command-Line Interface (interface en ligne de commande)
EMSI	École Marocaine des Sciences de l'Ingénieur
HTTP	HyperText Transfer Protocol
IP	Internet Protocol
IT	Information Technology (technologies de l'information)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAC	Media Access Control (adresse physique d'une interface réseau)
MCD	Modèle Conceptuel de Données
MVP	Minimum Viable Product (produit minimum viable)
OSI	Open Systems Interconnection (modèle de référence réseau)
OUI	Organizationally Unique Identifier (préfixe MAC identifiant le fabricant)
REST	Representational State Transfer (style d'architecture d'API web)
SPA	Single Page Application
SQL	Structured Query Language
TCP	Transmission Control Protocol
UML	Unified Modeling Language (langage de modélisation unifié)

Introduction Générale

Dans le cadre de la formation d'ingénieur en Informatique et Réseaux à l'École Marocaine des Sciences de l'Ingénieur (EMSI), les étudiants doivent effectuer un stage de perfectionnement en entreprise afin de mettre en pratique leurs connaissances théoriques dans un contexte professionnel réel. C'est dans ce cadre que ce stage de deux mois s'est déroulé au sein du département Informatique (IT) du Mövenpick Hôtel de Tanger.

Un réseau local d'hôtel voit se connecter, au fil du temps, un grand nombre d'équipements hétérogènes : postes de travail, imprimantes, caméras, bornes Wi-Fi et appareils invités. Sans outil dédié, le département IT ne dispose d'aucun moyen simple de savoir, à un instant donné, combien d'appareils sont actifs sur le réseau, ni d'identifier une connexion inhabituelle ou non autorisée. C'est pour répondre à ce besoin qu'est né le projet **NetSentry** : un outil interne de découverte et de cartographie des équipements connectés au réseau local, capable de scanner le réseau, de conserver un historique des détections et de signaler automatiquement tout équipement inconnu.

Le présent document constitue le rapport de stage du projet NetSentry. Il a pour objectif de présenter, de manière claire, structurée et détaillée, le contexte du stage, l'étude de l'existant et la méthodologie de travail suivie, la conception du système, son architecture technique et les technologies mobilisées, ainsi que les résultats obtenus.

Ce rapport s'organise en quatre chapitres.

Le **premier chapitre** présente le contexte général du projet : l'entreprise d'accueil, le département Informatique, la problématique à l'origine du projet et les objectifs poursuivis, ainsi qu'une étude des solutions déjà disponibles sur le marché et une analyse SWOT justifiant le choix d'une solution développée sur mesure.

Le **deuxième chapitre** détaille l'analyse des besoins – le cahier des charges qui a cadré le stage, ainsi que les exigences fonctionnelles et non fonctionnelles retenues – puis la conception du système : diagramme de cas d'utilisation, diagramme de séquence et modèle conceptuel de données.

Le **troisième chapitre** détaille l'architecture technique et les technologies mobilisées – notamment Nmap, cœur du moteur de scan – ainsi que l'organisation du travail retenue pour le projet (planning et suivi des tâches).

Le **quatrième chapitre** présente, par des captures d'écran, la mise en œuvre de l'outil : tableau de bord, détail d'un appareil, alertes, historique des scans et

versionnage GitHub.

Chapitre 1

Contexte général

1 Introduction

Ce chapitre présente le contexte général dans lequel s'inscrit ce stage : l'entreprise d'accueil, le département Informatique (IT) au sein duquel le stage s'est déroulé, la problématique à l'origine du projet NetSentry et les objectifs poursuivis, ainsi qu'une étude des solutions déjà disponibles sur le marché et une analyse SWOT justifiant le choix d'une solution développée sur mesure.

2 Organisme d'accueil

2.1 Présentation de l'entreprise

Le Mövenpick Hôtel de Tanger fait partie du groupe Mövenpick Hotels & Resorts, une chaîne hôtelière internationale opérant dans le secteur de l'hôtellerie et du tourisme. L'établissement propose des prestations d'hébergement, de restauration et d'organisation d'événements, destinées principalement à une clientèle d'affaires et de loisirs.

Le groupe Mövenpick a été fondé en 1948 à Zurich (Suisse) par Ueli Prager, initialement sous la forme d'un concept de restauration. Son activité hôtelière a débuté en 1973 avec l'ouverture de ses deux premiers hôtels à Zurich. Depuis septembre 2018, Mövenpick Hotels & Resorts est intégralement détenu par le groupe Accor, ce qui lui permet de bénéficier de son réseau de distribution et de ses outils opérationnels à l'échelle internationale. Le groupe est aujourd'hui présent dans plusieurs dizaines de pays, avec une forte présence en Europe et au Moyen-Orient.

L'hôtel de Tanger est structuré en plusieurs départements assurant chacun une fonction spécifique : la Direction Générale, les Ressources Humaines, la Finance, l'Hébergement (Front Office, Housekeeping), la Restauration, ainsi que le département Informatique (IT), au sein duquel s'est déroulé ce stage.

2.2 Environnement de l'entreprise

La clientèle du Mövenpick Hôtel de Tanger se compose principalement de voyageurs d'affaires, de touristes internationaux et de participants à des événements ou séminaires organisés au sein de l'établissement. L'hôtel fait par ailleurs appel à différents fournisseurs et prestataires externes, notamment pour la restauration, la maintenance des équipements, et la gestion des systèmes informatiques métiers (Property Management System – PMS), assurée par un prestataire spécialisé externe au département IT interne. Le marché hôtelier de Tanger est marqué par la présence de plusieurs établissements internationaux et locaux, notamment dans les segments haut de gamme et affaires ; le Mövenpick se positionne comme un établissement haut de gamme, misant sur la qualité de service et son emplacement stratégique dans la ville.

2.3 Département d'accueil et mission

Le département Informatique assure la gestion et la maintenance du système d'information de l'hôtel : réseau informatique, postes de travail, imprimantes et équipements divers. Les systèmes métiers principaux (PMS, point de vente) sont gérés par un prestataire externe spécialisé, ce qui recentre l'activité du département IT interne sur l'infrastructure réseau et le support technique. Il intervient de façon transversale auprès de l'ensemble des autres départements de l'hôtel et assure la disponibilité et la sécurité du réseau utilisé par le personnel.

Au sein de ce département, l'étudiant a été chargé de la conception et du développement d'un outil interne de découverte et de cartographie des équipements connectés au réseau informatique de l'hôtel, en complément des tâches de support courantes du département.

3 Contexte

Un réseau local d'hôtel voit se connecter, au fil du temps, un grand nombre d'équipements hétérogènes : postes fixes du personnel, imprimantes partagées, caméras de surveillance, bornes d'accès Wi-Fi, ainsi que des appareils invités (smartphones, ordinateurs portables) qui se connectent temporairement. Cette diversité et ce renouvellement permanent rendent difficile le maintien d'un inventaire fiable des équipements réellement présents sur le réseau à un instant donné.

Avant le déploiement de NetSentry, le département IT du Mövenpick Hôtel de Tanger ne disposait d'aucun outil de ce type : la connaissance du réseau reposait uniquement sur la mémoire des techniciens et sur des vérifications ponctuelles et manuelles, sans historique ni mécanisme d'alerte.

4 Problématique

L'absence d'un inventaire automatisé et à jour des équipements connectés pose plusieurs difficultés concrètes pour le département IT :

- Impossibilité de savoir, à un instant donné, combien d'appareils sont effectivement actifs sur le réseau et lesquels ;
- Absence de traçabilité : aucun historique ne permet de savoir quand un équipement est apparu, ni de repérer sa disparition ;
- Aucun mécanisme ne permet de détecter et de signaler la connexion d'un équipement inconnu ou potentiellement non autorisé ;
- Les vérifications, lorsqu'elles ont lieu, sont manuelles, ponctuelles et non reproductibles, ce qui les rend peu fiables sur la durée.

Ces limitations exposent le réseau interne à un risque de connexions non maîtrisées et compliquent la tâche du département IT dans le suivi de son propre parc d'équipements. Il devient donc nécessaire de développer un outil automatisé, léger et autonome, capable de scanner régulièrement le réseau, de conserver un historique exploitable, et de signaler toute anomalie de façon proactive.

5 Objectifs du projet

Le projet NetSentry vise à répondre à cette problématique à travers les objectifs suivants :

- **Découvrir** automatiquement les équipements connectés au réseau local et les identifier (adresse IP, adresse MAC, fabricant, ports ouverts) ;
- **Centraliser** ces informations dans une base de données persistante, consultable via un tableau de bord ;
- **Historiser** les scans afin de suivre l'apparition et la disparition des équipements dans le temps ;
- **Détecter et signaler** automatiquement tout équipement détecté pour la première fois, afin que l'administrateur IT puisse le vérifier et, le cas échéant, le classer comme connu ;
- **Automatiser** l'exécution des scans (planification périodique), sans intervention manuelle systématique ;
- **Déployer** la solution de façon conteneurisée (Docker), pour en faciliter l'installation et la portabilité sur un serveur de l'hôtel.

6 Solutions existantes

Plusieurs catégories d'outils permettent, à des degrés divers, de découvrir ou de superviser les équipements d'un réseau local :

6.1 Outils de scan en ligne de commande

- **Nmap** : scanner réseau open source, extrêmement puissant et précis (découverte d'hôtes, détection de ports et de services), mais qui reste un outil en ligne de commande : il ne conserve aucun historique entre deux exécutions et ne propose ni tableau de bord, ni mécanisme d'alerte natif.

6.2 Applications grand public

- **Fing** : application mobile de scan réseau simple d'utilisation, orientée particuliers, avec des fonctionnalités d'historique et d'alerte limitées à la version payante, et peu adaptée à un déploiement interne sur un serveur d'hôtel.
- **Advanced IP Scanner** : utilitaire Windows léger de découverte d'équipements, sans persistance des résultats ni planification automatique.

6.3 Solutions de supervision professionnelles

- **PRTG Network Monitor** : solution de supervision réseau complète et reconnue, mais commerciale, dont le coût de licence et la richesse fonctionnelle dépassent largement les besoins d'un simple inventaire d'équipements pour un département IT hôtelier.
- **Zabbix / Nagios** : plateformes de supervision open source très puissantes, mais dont l'installation, la configuration et la prise en main demandent une expertise et un temps d'appropriation importants – peu compatibles avec la contrainte de simplicité d'utilisation pour un personnel non spécialisé en réseau.

7 Limites observées

Cette étude fait ressortir un constat commun : les outils de scan bruts (Nmap) n'offrent aucune persistance ni interface, tandis que les solutions grand public manquent de fonctionnalités d'historisation et d'alerte adaptées, et que les plateformes de supervision professionnelles sont surdimensionnées, coûteuses ou trop complexes pour un déploiement léger et autonome au sein d'un seul département IT.

- Aucune solution ne combine à la fois simplicité de déploiement, auto-hébergement et historique exploitable pour ce périmètre restreint ;
- Les solutions commerciales impliquent un coût de licence récurrent, non justifié pour un besoin d'inventaire d'équipements ;
- Les solutions de supervision complètes nécessitent une expertise réseau que le personnel IT non spécialisé ne possède pas nécessairement ;
- Peu de solutions permettent un déploiement conteneurisé simple, cohérent avec les contraintes d'infrastructure de l'hôtel.

8 Analyse SWOT

Le tableau 1.1 synthétise les forces, faiblesses, opportunités et menaces identifiées pour le projet NetSentry.

Forces	Faiblesses
Développement sur mesure, adapté au besoin réel du département Basé sur Nmap, un outil de scan éprouvé et fiable Auto-hébergé et conteneurisé (Docker), sans coût de licence Interface simple, pensée pour du personnel non spécialisé en réseau	Projet réalisé par un seul stagiaire, sur une durée de deux mois Pas de détection de type d'équipement par apprentissage automatique OS detection (<code>nmap -O</code>) exclue du périmètre (nécessite des privilèges root supplémentaires) Historique GitHub encore synthétique à ce stade du projet
Opportunités	Menaces
Sensibilisation croissante des hôtels à la cybersécurité réseau Réutilisable pour d'autres hôtels du groupe ou d'autres PME Base pour des évolutions futures (notifications e-mail, classification par type d'appareil)	Évolution des équipements vers des adresses MAC aléatoires (confidentialité), limitant l'identification par fabricant Dépendance à un prestataire externe pour les systèmes métiers, hors périmètre du projet Réseau hôtelier partiellement hors du contrôle direct du département IT interne

TABLE 1.1 – Analyse SWOT du projet NetSentry

9 Méthodologie de travail

Le projet a été réalisé selon une approche itérative : mise à niveau sur les fondamentaux réseau, puis développement progressif du backend, du frontend, du module de détection d'anomalies, et enfin conteneurisation et documentation – chaque étape étant validée avant de passer à la suivante. L'organisation du travail au fil du stage (planning prévisionnel et suivi des tâches) est détaillée au chapitre 3.

10 Conclusion

Ce chapitre a posé le cadre général du stage – entreprise d'accueil, département Informatique, problématique et objectifs – et a montré, via l'étude de l'existant et

l'analyse SWOT, qu'aucune solution du marché ne répondait à ce besoin, justifiant une solution sur mesure. Le chapitre suivant présente l'analyse des besoins et la conception du système.

Chapitre 2

Analyse des besoins et conception

1 Introduction

Ce chapitre présente le cahier des charges qui a cadré le stage ainsi que les exigences fonctionnelles et non fonctionnelles retenues pour NetSentry, puis la modélisation du système qui en découle : le diagramme de cas d'utilisation, qui formalise les interactions entre les acteurs et le système, le diagramme de séquence, qui détaille le déroulement complet d'un scan, et le modèle conceptuel de données, qui structure les informations persistées en base.

2 Cahier des charges

Le cadrage du projet a été formalisé dans un cahier des charges remis en début de stage, définissant le contexte, les objectifs, le périmètre fonctionnel et non fonctionnel, ainsi que les contraintes et livrables attendus.

2.1 Livrables attendus

- Code source de l'application (backend, frontend, scripts de scan) ;
- Base de données structurée contenant l'historique des équipements détectés ;
- Tableau de bord fonctionnel de visualisation ;
- Image(s) Docker et instructions de déploiement ;
- Documentation technique et rapport de stage détaillant la démarche, les choix techniques et les résultats obtenus.

2.2 Contraintes

Le projet devait être réalisé dans un délai de deux mois, avec un accès limité aux systèmes métiers de l'hôtel (gérés par un prestataire externe). Le scan réseau ne pouvait être exécuté que sur le réseau interne de l'hôtel, avec l'autorisation du

département IT, et la solution devait privilégier une base de données relationnelle (SQL) en cohérence avec les outils disponibles côté hôtel.

3 Exigences fonctionnelles

Les exigences fonctionnelles traduisent les besoins identifiés en fonctionnalités concrètes pour NetSentry :

- **Découverte des équipements** : scanner le réseau local afin d'identifier tous les équipements actuellement connectés (adresse IP, adresse MAC).
- **Identification des équipements** : déterminer, lorsque c'est possible, le fabricant (via correspondance MAC/OUI) et les ports/services ouverts.
- **Historique des équipements** : enregistrer chaque scan en base de données afin de conserver un historique (nouveaux équipements, équipements connus, équipements disparus).
- **Tableau de bord** : visualiser en temps réel la liste des équipements connectés, leur statut, et l'historique d'activité du réseau.
- **Détection d'anomalies** : identifier et signaler automatiquement tout équipement inconnu apparaissant pour la première fois sur le réseau.
- **Alertes** : notifier l'administrateur IT en cas de détection d'un équipement non reconnu, et lui permettre de l'acquitter ou de le classer comme connu.
- **Planification** : exécuter les scans automatiquement selon une fréquence configurable, sans intervention manuelle systématique.
- **Déploiement** : conteneuriser l'application avec Docker afin de permettre un déploiement simple sur un serveur de l'hôtel.

4 Exigences non fonctionnelles

Les exigences non fonctionnelles garantissent la qualité, la sécurité et la performance du système, indépendamment de ses fonctionnalités métier :

- **Ergonomie** : le tableau de bord doit rester simple et lisible pour un personnel non spécialisé en réseau.
- **Sécurité** : l'accès au tableau de bord doit être protégé par authentification (JWT), et les mots de passe stockés de façon chiffrée (hachage bcrypt).
- **Confidentialité** : aucune donnée du réseau interne ne doit être transmise à des services externes – l'identification des fabricants s'appuie sur une base OUI locale, sans appel API externe.
- **Autonomie** : les scans doivent pouvoir s'exécuter de façon planifiée, sans intervention manuelle systématique.
- **Portabilité** : l'application doit être conteneurisée (Docker) afin de pouvoir être déployée sur différents environnements.

- **Performance** : un scan complet du réseau local doit s'exécuter en un temps raisonnable, sans perturber le trafic réseau existant (usage d'un délai d'expiration par hôte).
- **Maintenabilité** : le code doit être structuré (architecture claire backend/frontend), documenté et versionné (Git).

5 Diagramme de cas d'utilisation

Le système NetSentry expose un seul rôle applicatif : un **administrateur IT** authentifié, qui pilote le tableau de bord, déclenche les scans et traite les alertes. Un second acteur, purement système, intervient également : le **planificateur (cron)**, qui déclenche automatiquement les scans planifiés sans intervention humaine.

Toutes les actions accessibles à l'administrateur IT – à l'exception de l'authentification elle-même – nécessitent une session valide : chacune d'elles inclut donc (relation *include*) le cas d'utilisation **S'authentifier**, qui repose sur la vérification des identifiants et l'émission d'un jeton JWT. C'est également le cas des deux cas d'utilisation qui déclenchent un scan (*Lancer un scan manuel* et *Exécuter un scan planifié*), qui incluent tous deux le sous-comportement **Détecter les appareils (Nmap)**, commun aux deux modes de déclenchement. La figure 2.1 présente l'ensemble de ces cas d'utilisation et de leurs relations.

Comme le montre la figure 2.1, l'administrateur IT peut consulter le tableau de bord, lancer un scan manuel, consulter l'historique des scans, consulter les appareils détectés et le détail d'un appareil, marquer un appareil comme connu, ainsi que consulter et acquitter les alertes. Le planificateur (cron), quant à lui, ne déclenche que l'exécution des scans planifiés, sans passer par l'authentification puisqu'il s'agit d'un acteur système interne au serveur.



FIGURE 2.1 – Diagramme de cas d'utilisation de NetSentry : l'authentification est incluse par l'ensemble des actions de l'administrateur IT

6 Diagramme de séquence

Le scénario le plus représentatif du fonctionnement de NetSentry est l'exécution d'un scan, depuis son déclenchement par l'administrateur IT jusqu'à la détection éventuelle d'une anomalie. Le diagramme de séquence de la figure 2.2 détaille ce scénario de bout en bout, à travers les cinq composants impliqués : l'interface React, l'API backend (Node.js/Express), le service Nmap, la base PostgreSQL et le service d'anomalies.

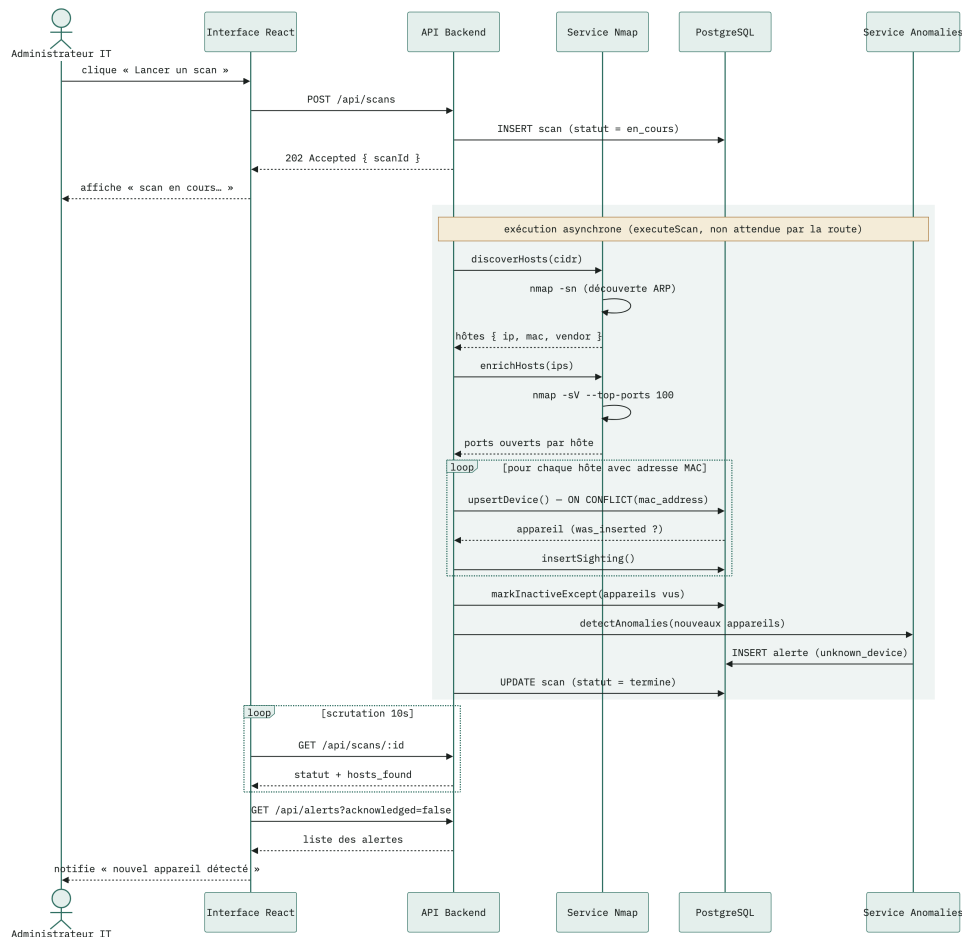


FIGURE 2.2 – Diagramme de séquence – exécution d'un scan et détection d'une anomalie

Le déclenchement d'un scan (`POST /api/scans`) répond immédiatement (`202 Accepted`) sans attendre la fin de l'exécution, qui se poursuit de façon asynchrone : découverte des hôtes actifs par ARP (`nmap -sn`), puis enrichissement des hôtes découverts par une énumération de ports (`nmap -sV -top-ports 100`). Pour chaque équipement détecté possédant une adresse MAC, le backend met à jour la table des équipements (`upsertDevice`, avec détection d'insertion via `ON CONFLICT`) et jour-

nalise l’observation (`insertSighting`) ; les équipements non revus lors du scan sont marqués inactifs, et le service d’anomalies génère une alerte pour tout équipement nouvellement inséré. L’interface React scrute ensuite périodiquement (*polling* toutes les 10 secondes) le statut du scan et la liste des alertes non acquittées.

7 Modélisation des données (MCD)

Le modèle conceptuel de données de la figure 2.3 définit les entités et relations utilisées pour stocker l’historique des équipements détectés, conformément au schéma implémenté dans PostgreSQL.

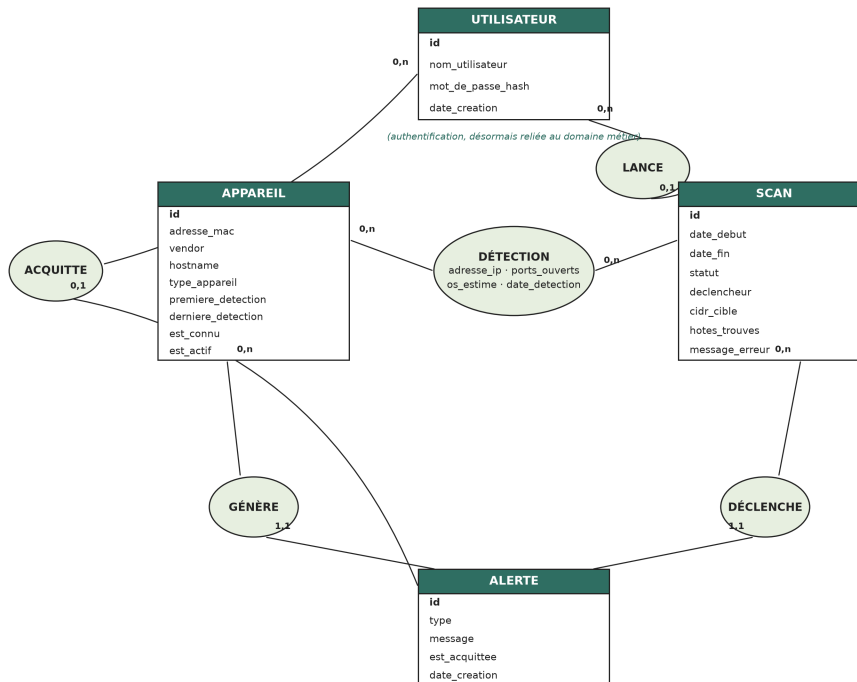


FIGURE 2.3 – Modèle conceptuel de données (MCD) de NetSentry

Le modèle distingue quatre entités principales. L’entité **APPAREIL** représente un équipement, identifié par son adresse MAC – stable dans le temps, contrairement à l’adresse IP attribuée par DHCP – et porte son statut courant (`est_connu`, `est_actif`). L’entité **SCAN** représente une exécution du moteur de scan (manuelle ou planifiée), avec son statut et sa cible (CIDR). L’association **DÉTECTION**, de cardinalité (0,n)–(0,n) entre **APPAREIL** et **SCAN**, constitue l’historique append-only consultable dans le tableau de bord : chaque détection porte l’adresse IP observée, les ports ouverts et la date de détection pour cette occurrence précise du scan. L’entité **ALERTE** est générée soit par la détection d’un nouvel appareil (association

GÉNÈRE), soit par le scan qui l'a révélée (association **DÉCLENCHÉ**), et peut être acquittée par l'administrateur IT.

Enfin, l'entité **UTILISATEUR**, dédiée à l'authentification, est rattachée au domaine métier par deux associations : **LANCE** (0,n – 0,1), qui trace l'utilisateur à l'origine d'un scan déclenché manuellement (un scan planifié par le *cron* n'a naturellement aucun utilisateur associé), et **ACQUITTE** (0,n – 0,1), qui trace l'utilisateur ayant acquitté une alerte. Ces deux associations, implémentées par les colonnes nulables `triggered_by_user_id` sur `scans` et `acknowledged_by_user_id` sur `alerts`, apportent une traçabilité utile même dans un contexte à administrateur unique, et prendront tout leur sens si plusieurs comptes IT venaient à être créés.

8 Conclusion

Ce chapitre a détaillé les exigences qui ont cadré le projet NetSentry, puis a formalisé les interactions et les données du système à travers trois diagrammes complémentaires. Le chapitre suivant détaille l'architecture technique retenue, les technologies mobilisées – au premier rang desquelles Nmap – ainsi que l'organisation du travail durant le stage.

Chapitre 3

Architecture technique et technologies

1 Introduction

Ce chapitre présente l'architecture technique de NetSentry ainsi que les technologies mobilisées pour sa réalisation. Une attention particulière est portée à **Nmap**, l'outil au cœur du moteur de scan, dont le fonctionnement conditionne l'ensemble de la chaîne de découverte réseau. Le chapitre se conclut par l'organisation du travail retenue durant le stage : planning prévisionnel et suivi des tâches.

2 Nmap : l'outil au cœur du moteur de scan

2.1 Présentation

Nmap (*Network Mapper*) est un scanner réseau open source créé en 1997 par Gordon Lyon (connu sous le pseudonyme *Fyodor*). Distribué gratuitement pour la plupart des systèmes d'exploitation, il est devenu la référence des outils de découverte réseau, aussi bien dans un usage d'administration système que dans un cadre de sécurité offensive ou défensive. NetSentry s'appuie directement sur le binaire **nmap**, invoqué depuis le backend Node.js via un processus enfant (`child_process.spawn`), plutôt que sur une bibliothèque tierce – ce qui garantit un contrôle précis des arguments passés et évite tout risque d'injection de commande (les arguments sont transmis sous forme de tableau, jamais construits par concaténation de chaînes).



FIGURE 3.1 – Logo du projet Nmap (nmap.org)

2.2 Découverte d'hôtes par ARP

Sur un réseau local (LAN), Nmap privilégie le protocole **ARP** (*Address Resolution Protocol*) plutôt que de simples requêtes *ping* (ICMP) pour découvrir les hôtes actifs. Une requête ARP est diffusée en broadcast sur le segment local (à qui possède l'adresse IP $X?z$) ; chaque machine active y répond directement avec son adresse MAC. Cette méthode est à la fois plus fiable qu'un ping – de nombreux pare-feux bloquent l'ICMP mais laissent passer l'ARP, indispensable au fonctionnement même du réseau local – et plus riche : elle fournit *gratuitement* l'adresse MAC de chaque hôte, sans scan de port supplémentaire. C'est précisément cette adresse MAC que NetSentry utilise ensuite comme identifiant stable d'un équipement (une adresse IP, attribuée par DHCP, peut changer d'un scan à l'autre).

2.3 Les deux phases du scan NetSentry

Le moteur de scan de NetSentry exécute Nmap en deux temps distincts, illustrés au diagramme de séquence du chapitre précédent (figure 2.2) :

1. **Découverte** (`nmap -sn <CIDR>`) : un scan *ping-only*, sans énumération de ports, qui identifie rapidement l'ensemble des hôtes actifs sur la plage réseau cible (par exemple `192.168.1.0/24`) ainsi que leur adresse MAC et, lorsque Nmap parvient à le déterminer, le fabricant associé.
2. **Enrichissement** (`nmap -sV --top-ports 100`) : un second scan, exécuté uniquement sur les hôtes découverts à l'étape précédente et regroupés en un seul processus (plutôt qu'un processus par hôte), qui identifie les ports ouverts parmi les 100 ports les plus courants ainsi que les services et versions associés.

```
# Phase 1 - decouverte (ARP, sans scan de port)
nmap -sn 192.168.1.0/24 -oX -

# Phase 2 - enrichissement (ports + services, hotes
# decouverts uniquement)
nmap -sV --top-ports 100 -oX - 192.168.1.14 192.168.1.23 ...
```

Listing 3.1 – Exemple des deux commandes Nmap exécutées par NetSentry

Ce découpage en deux phases répond à un objectif de performance : limiter l'énumération de ports – l'opération la plus coûteuse en temps – aux seuls hôtes réellement présents, plutôt que de la tenter sur l'ensemble de la plage d'adresses. Un délai d'expiration par hôte (`-host-timeout`) est également appliqué, afin de garantir qu'un hôte injoignable ne bloque pas l'ensemble du scan.

2.4 Choix hors périmètre

La détection de système d'exploitation (`nmap -O`) a été volontairement exclue du périmètre du projet : elle nécessite des privilèges supplémentaires et un accès direct aux sockets bruts, avec une fiabilité variable selon les équipements, ce qui la rend peu adaptée à une exécution automatisée et non supervisée. Pour la même raison, l'exécution de Nmap au sein du conteneur Docker requiert l'octroi explicite des capacités Linux `NET_ADMIN` et `NET_RAW` (construction de paquets ARP bruts), configurées dans `docker-compose.yml` via `cap_add`.

2.5 Identification du fabricant (OUI)

L'adresse MAC d'un équipement encode, dans ses trois premiers octets, un identifiant de fabricant appelé **OUI** (*Organizationally Unique Identifier*). NetSentry résout ce préfixe via une base de données OUI statique embarquée (bibliothèque `oui`), sans appel à un service externe – ce qui garantit qu'aucune information sur les équipements du réseau interne ne quitte le serveur, conformément à l'exigence de confidentialité du chapitre précédent. Un cas particulier mérite d'être signalé : les smartphones modernes (iOS et Android) génèrent par défaut une adresse MAC *aléatoire* par réseau Wi-Fi rejoint, à des fins de confidentialité. Une telle adresse est dite *localement administrée* (le second bit du premier octet est positionné à 1) et ne correspond à aucun fabricant réel : NetSentry le détecte (`first_byte & 0x02`) et affiche alors un fabricant *inconnu*, non par échec de résolution, mais parce que l'adresse elle-même n'est pas une adresse constructeur authentique.

3 Autres technologies utilisées

3.1 Backend

Node.js et Express

Node.js est un environnement d'exécution JavaScript côté serveur, adapté au traitement d'opérations asynchrones – typiquement l'attente de la fin d'un scan Nmap sans bloquer le reste de l'application. Express, framework minimaliste construit sur Node.js, structure l'API REST de NetSentry (routes d'authentification, de scans, d'équipements et d'alertes) et la génération des jetons JWT.



(a) Node.js



(b) Express

FIGURE 3.2 – Technologies du backend NetSentry

3.2 Base de données

PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source, réputé pour sa robustesse et le respect strict des propriétés ACID. NetSentry l'utilise pour persister les équipements, l'historique des détections (table append-only), les scans et les alertes, avec des types natifs adaptés au domaine (`MACADDR`, `INET`, `JSONB`).

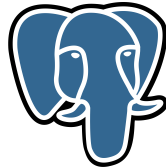


FIGURE 3.3 – Logo PostgreSQL

3.3 Frontend

React

React est une bibliothèque JavaScript pour la construction d'interfaces utilisateur par composants. Le tableau de bord NetSentry (liste des équipements, détail d'un appareil, alertes, historique des scans) est développé en React, avec des mises à jour périodiques (*polling*) de l'état des scans et des alertes.

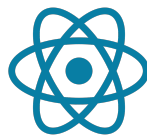


FIGURE 3.4 – Logo React

3.4 Conteneurisation et versionnage

Docker

Docker permet d'empaqueter chacun des trois services de NetSentry (backend, frontend, base de données) dans des conteneurs isolés mais orchestrés ensemble via Docker Compose, garantissant un déploiement reproductible sur le serveur de l'hôtel, indépendamment de son environnement d'installation.



FIGURE 3.5 – Logo Docker

Git

Le projet est versionné avec Git et hébergé sur un dépôt GitHub public, permettant de suivre l'évolution du code source tout au long du stage.



FIGURE 3.6 – Logo Git

4 Architecture du système

4.1 Architecture en couches

NetSentry suit une architecture en trois couches, illustrée à la figure 3.7 :

- **Couche de scan** : le service Nmap, exécuté avec les privilèges réseau nécessaires (`cap_add: NET_ADMIN, NET_RAW`), interroge périodiquement les équipements du réseau local ;
- **Couche logique métier** : l'API backend (Node.js/Express) orchestre les scans, applique la logique de détection d'anomalies et expose une API REST authentifiée (JWT) ;
- **Couche de persistance et de présentation** : PostgreSQL stocke l'historique, et le tableau de bord React (servi par Nginx en production) est consulté par l'administrateur IT depuis son navigateur.

Le déploiement cible étant un serveur Linux, le service backend utilise `network_mode: host` dans `docker-compose.yml`, afin que le scanner Nmap puisse voir le réseau local réel de l'hôtel plutôt que le seul réseau interne (isolé) de Docker – une contrainte indispensable au bon fonctionnement de la découverte ARP en conteneur.

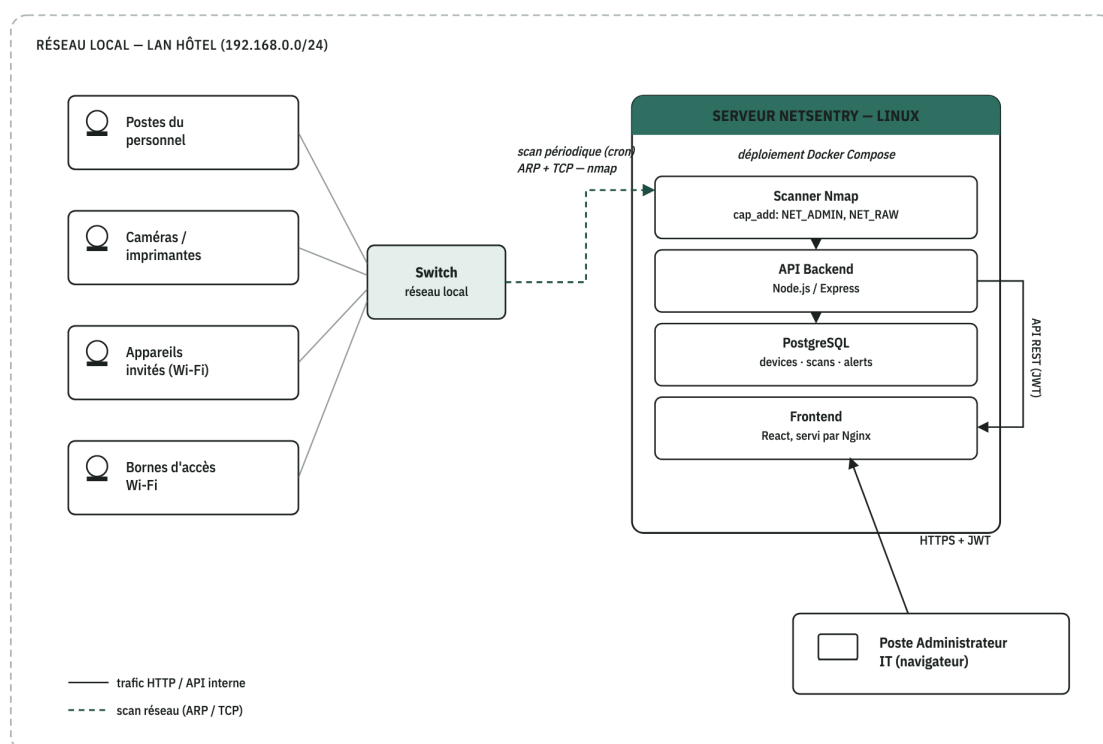


FIGURE 3.7 – Architecture générale de NetSentry et intégration au réseau local de l'hôtel

5 Planification du projet

5.1 Diagramme de Gantt

Le planning prévisionnel du stage, tel que défini dans le cahier des charges, répartit les huit semaines du stage en cinq grandes phases : prise en main de l'environnement réseau, conception et développement du backend, développement du tableau de bord, implémentation de la détection d'anomalies, puis conteneurisation et finalisation. La figure 3.8 illustre ce planning sous forme de diagramme de Gantt.

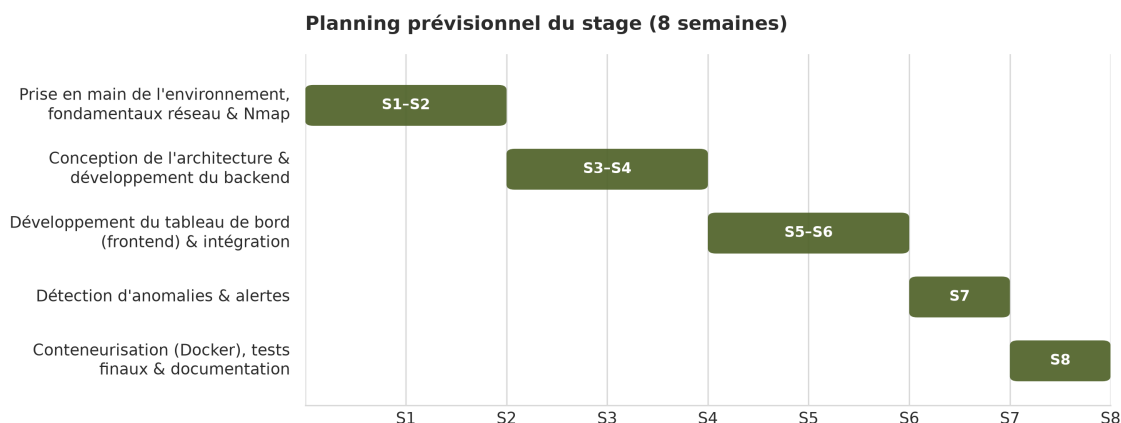


FIGURE 3.8 – Diagramme de Gantt du planning prévisionnel du stage

5.2 Organisation du travail (Kanban)

Le suivi de l'avancement des tâches s'est appuyé sur une approche inspirée de Kanban, avec un tableau organisé en trois colonnes (à faire, en cours, terminé) permettant de visualiser l'état des différentes tâches du projet au fil du stage. La figure 3.9 présente l'état de ce tableau en fin de stage.

Organisation du travail (méthode Kanban) — état du tableau en fin de stage



FIGURE 3.9 – Organisation du travail selon la méthode Kanban

6 Conclusion

Ce chapitre a détaillé le fonctionnement de Nmap, cœur technique du projet, ainsi que l'ensemble des technologies et l'architecture retenues pour NetSentry, et l'organisation du travail suivie durant le stage. Le chapitre suivant présente la mise en œuvre concrète de l'outil à travers des captures d'écran.

Chapitre 4

Mise en œuvre et captures d'écran

1 Introduction

Ce chapitre présente, à travers des captures d'écran, le fonctionnement réel de l'outil NetSentry développé durant le stage, à partir d'un scan effectué sur un réseau local de test : tableau de bord, détail d'un appareil, alertes, historique des scans, ainsi que l'historique de versionnage du projet sur GitHub.

2 Tableau de bord

Le tableau de bord constitue le point d'entrée principal de l'application. Il présente l'ensemble des appareils détectés sur le réseau, leur statut (en ligne / hors ligne), le nombre total d'appareils ainsi que le nombre d'appareils non encore vérifiés (*unreviewed*), et permet de déclencher un scan manuel en un clic.

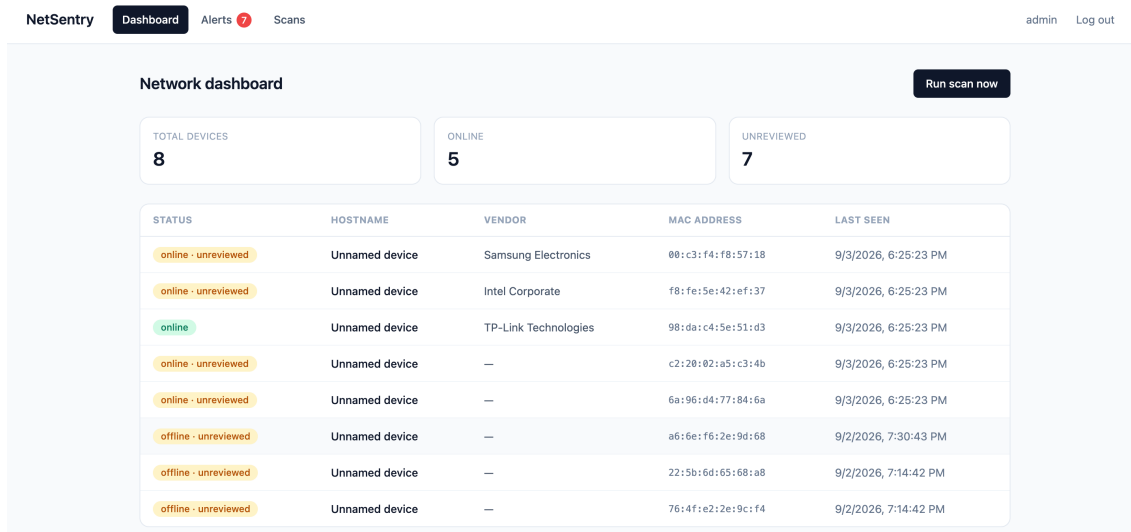


FIGURE 4.1 – Tableau de bord : vue d'ensemble des appareils détectés sur le réseau

3 Détail d'un appareil

Le détail d'un appareil retrace son historique de détection au fil des scans successifs (adresse IP observée, ports ouverts, date de détection), et permet à l'administrateur IT de le marquer comme connu (liste blanche), ce qui met fin aux alertes futures pour cet équipement.

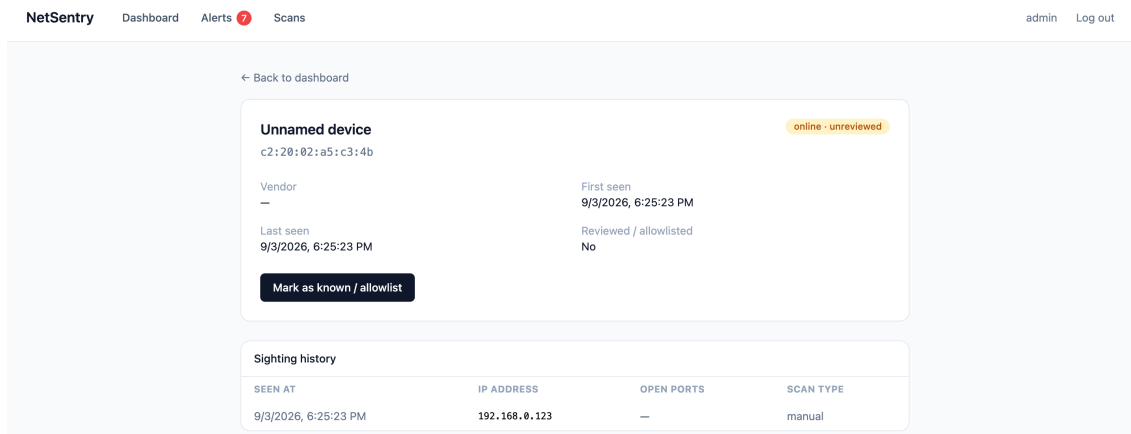


FIGURE 4.2 – Détail d'un appareil : historique de détection et mise en liste blanche

4 Alertes

La page des alertes liste les appareils non reconnus signalés après un scan. Conformément à la politique retenue, une alerte est générée une seule fois, lors de la première détection d'un appareil inconnu ; l'administrateur IT peut ensuite l'acquitter ou classer l'appareil comme connu pour désactiver les alertes futures le concernant.

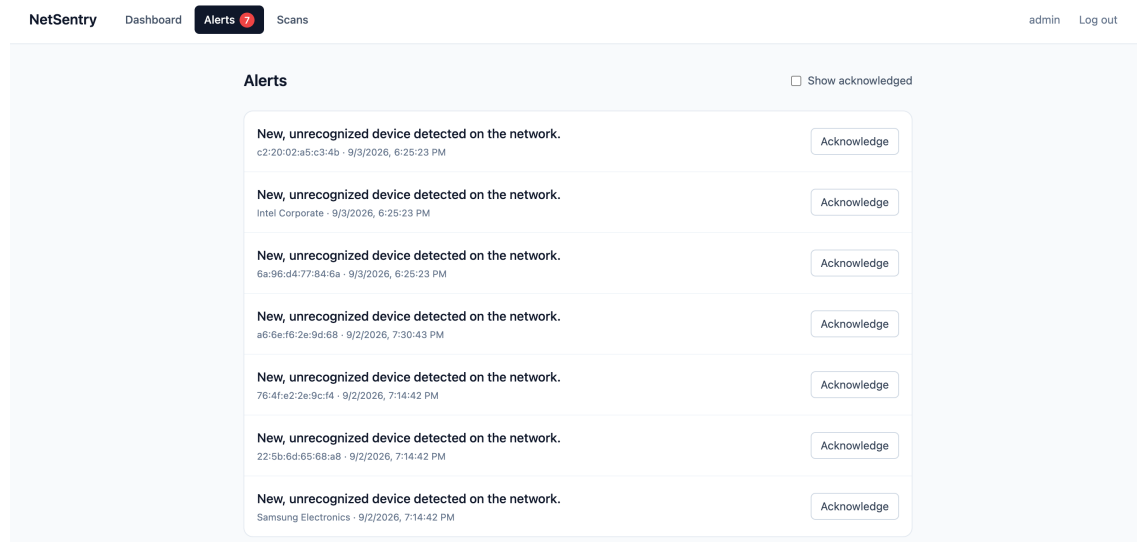


FIGURE 4.3 – Alertes : appareils non reconnus signalés après un scan

5 Historique des scans

L'historique des scans conserve la trace de chaque exécution, manuelle ou planifiée, avec son statut, le nombre d'hôtes trouvés et, le cas échéant, le message d'erreur associé.

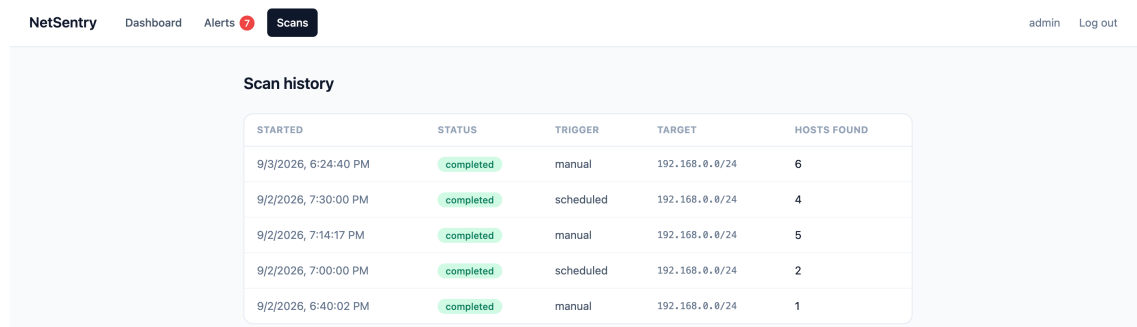


FIGURE 4.4 – Historique des scans exécutés (manuels et planifiés)

6 Versionnage

Le projet est versionné avec Git et hébergé sur un dépôt GitHub public. L'historique reste volontairement synthétique à ce stade du stage : un commit initial, l'ajout des documents de cadrage, puis un commit unique regroupant la construction du MVP (backend, frontend, conteneurisation). Il sera affiné par des commits plus granulaires au fil des prochaines itérations.

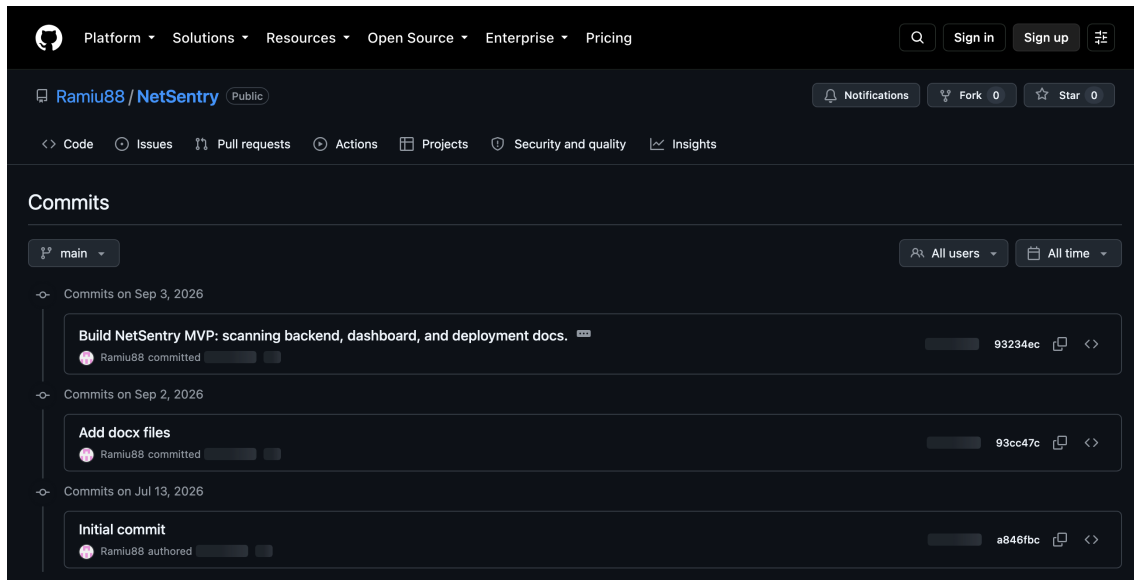


FIGURE 4.5 – Historique des commits du dépôt GitHub NetSentry

7 Conclusion

Ces captures d'écran illustrent le fonctionnement réel et vérifié de la chaîne complète de NetSentry, du scan réseau jusqu'à la restitution dans le tableau de bord. Elles constituent la démonstration concrète des objectifs fixés au premier chapitre.

Conclusion Générale

Ce rapport a présenté les travaux réalisés durant le stage de perfectionnement effectué au sein du département Informatique (IT) du Mövenpick Hôtel de Tanger, portant sur la conception et le développement de **NetSentry**, un outil interne de découverte et de cartographie des équipements connectés au réseau local de l'hôtel.

1 Bilan des réalisations

Le premier chapitre a permis de poser le contexte général du stage et du projet, en mettant en évidence l'absence, au sein du département IT, d'un inventaire fiable des équipements connectés au réseau, et a montré, à travers une étude de l'existant et une analyse SWOT, qu'aucune solution disponible sur le marché ne combinait simplicité de déploiement, auto-hébergement et historique exploitable pour ce périmètre restreint, justifiant ainsi le développement d'une solution sur mesure. Le deuxième chapitre a détaillé les exigences retenues puis formalisé la conception du système à travers ses diagrammes UML. Le troisième chapitre a détaillé l'architecture technique retenue – avec Nmap comme moteur de scan – ainsi que les technologies mobilisées. Le quatrième chapitre a enfin illustré, par des captures d'écran, le fonctionnement réel et vérifié de l'outil : découverte des équipements, tableau de bord, alertes, historique des scans et déploiement conteneurisé.

2 Défis rencontrés et solutions apportées

Plusieurs défis ont marqué la réalisation de ce projet :

- **Périmètre accessible limité** : les systèmes métiers de l'hôtel (PMS, point de vente) étant gérés par un prestataire externe, le sujet initial a dû être recentré sur le seul réseau interne, réellement accessible au département IT.
- **Fiabilité de l'identification des équipements** : certains appareils (notamment les smartphones modernes) utilisent des adresses MAC aléatoires à des fins de confidentialité, rendant leur fabricant impossible à déterminer – une limitation inhérente à toute identification basée sur l'adresse MAC, documentée plutôt que masquée dans l'outil.

- **Contrainte de simplicité** : concevoir un tableau de bord exploitable par du personnel non spécialisé en réseau, tout en s'appuyant sur un outil aussi riche que Nmap, a nécessité de limiter volontairement certaines fonctionnalités avancées (détection de système d'exploitation notamment).

3 Compétences acquises et renforcées

Ce stage a permis de renforcer plusieurs compétences clés :

- **Compétences réseau** : adressage IP/MAC, protocole ARP, notion de ports, et prise en main approfondie de Nmap.
- **Compétences techniques** : développement backend (Node.js, Express, PostgreSQL), frontend (React), conteneurisation (Docker), et sécurisation d'une API (authentification JWT).
- **Modélisation et architecture** : conception UML (cas d'utilisation, séquence, modèle conceptuel de données) et mise en œuvre d'une architecture en couches.
- **Compétences transversales** : compréhension du fonctionnement d'un département informatique en entreprise et de sa collaboration avec un prestataire externe, autonomie et adaptation du périmètre d'un projet aux contraintes réelles d'une organisation.

4 Perspectives d'évolution

Le système développé constitue une base solide pour plusieurs évolutions futures :

- **Notifications par e-mail** : informer automatiquement l'administrateur IT dès la détection d'un équipement inconnu, en complément de l'alerte affichée dans le tableau de bord.
- **Classification par type d'appareil** : enrichir l'identification des équipements par une heuristique combinant fabricant et ports ouverts.
- **Historique Git plus granulaire** : poursuivre le versionnage du projet par des commits plus fréquents au fil des prochaines itérations.
- **Déploiement en production** : installation de NetSentry sur le serveur définitif de l'hôtel, dans les conditions réelles d'exploitation du réseau.

En conclusion, ce stage a représenté une étape importante dans le parcours de l'étudiant, en le confrontant aux réalités d'un développement logiciel mené en autonomie, à la croisée du développement web et de l'administration réseau, au service d'un besoin réel exprimé par une entreprise d'accueil.

Bibliographie

- [1] OpenJS Foundation, “Documentation officielle node.js.” <https://nodejs.org/en/docs>, 2026. Consulté durant le stage.
- [2] OpenJS Foundation, “Express.js – documentation officielle.” <https://expressjs.com/>, 2026. Consulté durant le stage.
- [3] Docker Inc., “Documentation officielle docker.” <https://docs.docker.com/>, 2026. Consulté durant le stage.
- [4] G. Lyon, “Nmap reference guide.” <https://nmap.org/book/man.html>, 2026. Consulté durant le stage.
- [5] Mozilla Contributors, “Mdn web docs – javascript.” <https://developer.mozilla.org/fr/docs/Web/JavaScript>, 2026. Consulté durant le stage.
- [6] Mövenpick Hotels & Resorts, “Site officiel mövenpick hotels & resorts.” <https://movenpick.accor.com/>, 2026. Consulté durant le stage.
- [7] Accor, “Communiqué de presse relatif à l’acquisition de mövenpick hotels & resorts.” <https://press.accor.com>, 2018. Consulté durant le stage.
- [8] The PostgreSQL Global Development Group, “Postgresql documentation.” <https://www.postgresql.org/docs/>, 2026. Consulté durant le stage.
- [9] Meta Platforms, Inc., “React – documentation officielle.” <https://react.dev/>, 2026. Consulté durant le stage.